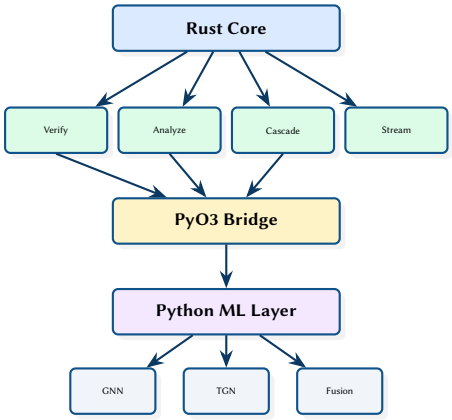
NetAssure: Real-Time Network Analytics Platform

GNN-Based Anomaly Detection, Formal Verification, and Cascade Simulation

Simon Knight

March 2026 • Version 0.1.0

Last updated: March 12, 2026



Abstract. This report presents NetAssure, a hybrid Rust/Python network analytics platform that provides six complementary analysis paradigms: formal verification via Header Space Analysis, graph-theoretic metrics, Monte Carlo failure cascade simulation, GNN-based anomaly detection, real-time streaming with WebSocket alerting, and topology optimisation. The Rust core handles performance-critical deterministic algorithms (topology ingestion, verification, graph analysis, cascade simulation), while a Python layer provides PyTorch-based machine learning including temporal graph networks and multi-modal fusion scoring. PyO3 bindings bridge the two worlds, exposing verification, analysis, and cascade simulation to the Python ML pipeline.

Contents

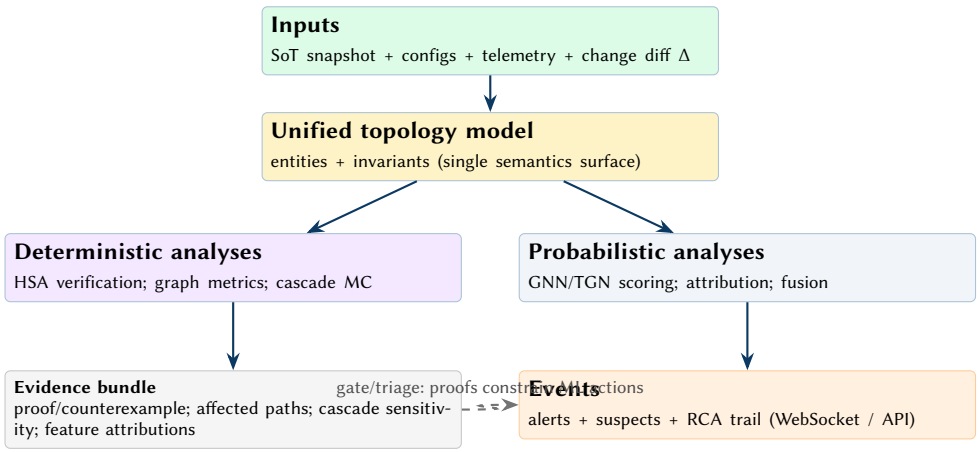


Figure 1. Conceptual control loop for multi-paradigm analysis. NetAssure treats verification artifacts as constraints (what must be true) and ML outputs as advisory signals (what is likely true). The operator-facing output is not a single score, but an evidence bundle and a bounded RCA trail.

1 Introduction and Motivation

Network operators need multiple analysis perspectives to understand complex topologies: formal verification alone cannot predict cascade failures, while machine learning alone cannot guarantee reachability invariants. NetAssure integrates six complementary paradigms into a single platform with a unified topology model.

The six paradigms are:

1. **Formal Verification** — Header Space Analysis (HSA), reachability proofs, loop detection, and traffic isolation checking.
2. **Graph Algorithms** — Centrality, community detection, path diversity, and robustness metrics.
3. **Failure Cascade Modelling** — Monte Carlo simulation with load redistribution and scenario comparison.
4. **Machine Learning (ML)/Graph Neural Network (GNN)** — Anomaly detection, temporal graph networks, and feature attribution via GNNExplainer.
5. **Real-Time Streaming** — Inference pipeline, anomaly detector, alert engine, and WebSocket streaming.
6. **Optimisation** — Critical node identification and redundancy recommendations.

: Hybrid Rust/Python Architecture

Deterministic algorithms (verification, graph metrics, cascade simulation) are implemented in Rust for performance and correctness. Machine learning workloads run in Python (PyTorch/PyTorch Geometric) where the ecosystem is mature. PyO3 bridges the two at the function call level, avoiding serialisation overhead.

1.1 Related Work and Positioning

NetAssure is positioned at the intersection of (i) network configuration verification and policy analysis, (ii) resilience modelling via cascade simulation, and (iii) ML-based anomaly detection on graph-structured telemetry.

Verification and policy analysis. Header Space Analysis (HSA) provides the basis for scalable reachability and isolation checking by representing forwarding behaviour as transformations over header subspaces [1, 2]. Tools such as VeriFlow focus on real-time invariant checking by intercepting control plane updates and validating changes before installation [3]. Batfish models routing and forwarding to answer reachability and security questions over configuration snapshots and what-if scenarios [4]. NetAssure adopts the HSA-style view for formal verification, but explicitly couples it with graph analytics and operational resilience modelling, so operators can explain not only *whether* traffic can flow, but also *how robust* that flow is under failures.

Network programming and semantics. NetKAT provides a rigorous semantic foundation for reasoning about packet-processing functions and composing policies [5]. While NetAssure is not a network programming language, it borrows the discipline of explicit semantics: verification and analysis operate over a single topology model and a consistent set of invariants.

Resilience and cascading failures. Cascading failure models are widely used to study how local faults can amplify into systemic outages in interdependent systems. Motter and Lai show how load redistribution can

lead to abrupt breakdowns under targeted perturbations [6]. NetAssure instantiates this idea at the topology level with a round-based simulator, enabling Monte Carlo estimation of cascade depth and scope and explicit before/after comparisons when proposing redundancy.

Graph ML for anomaly detection. GNNs such as GCNs [7], GraphSAGE [8], and GAT [9] support learning from relational structure, while Temporal Graph Networks extend this capability to dynamic event streams [10]. For operator trust, explanation techniques such as GNNExplainer can attribute anomalies to influential subgraphs and features [11]. NetAssure integrates these models into an online pipeline, but keeps probabilistic outputs behind a trust boundary (Section 2) and uses deterministic engines for assertions and guardrails.

: Principled Hybrid: Proofs + Predictions

NetAssure treats verification results as hard constraints and ML results as ranked hypotheses. When the two disagree, the system surfaces the disagreement explicitly rather than collapsing it into a single score.

1.2 Comparison to Other Tools

Tables 1 and 2 summarise how NetAssure compares to common open-source, academic, and commercial tooling. The intent is not to declare a universal winner, but to clarify trade-offs and where NetAssure provides an end-to-end workflow that spans configuration semantics, topology reasoning, resilience modelling, and online inference.

Table 1. Comparison: verification and analysis tools.

Tool	Primary role	Strengths	Weaknesses	Typical usage
NetAssure	Unified topology analysis	Combines HSA, graph metrics, cascade simulation, and ML scoring behind a single topology model	Young project; depends on good topology+telemetry ingestion	Ops: correctness + resilience + RCA
Batfish [4]	Config Q&A	Strong semantics for routing/ACL questions over snapshots	Not streaming-first; less focus on resilience modelling	Pre-change validation / what-if Q&A
VeriFlow [3]	Inline invariants	Real-time checks on control-plane updates	Narrower scope than full analytics stack	Guardrail on updates
NetKAT [5]	Policy semantics	Rigorous foundations for composition/equivalence	Not an operational runtime	Research / policy compilation
Minesweeper [12]	BGP verification	Scales to interdomain policy/config correctness	Focused on BGP, not internal telemetry analytics	ISP BGP safety checks
ARC / Deltanet (placeholder) [13, 14]	Verification frameworks	Unified/incremental verification under change	Placeholder citations here	Continuous verification workflows

Table 2. Comparison: inventory, automation, monitoring, and observability stack.

Tool	Primary role	What it provides	How NetAssure fits
NetBox [15]	Source of truth	Inventory + relationships + intent metadata	Authoritative input for topology ingestion
NAPALM [16] / Netmiko [17]	Device access	Multi-vendor state/config collection	Feed snapshots + confirm post-change state
Ansible [18]	Automation	Desired state workflows	Validate diffs before/after rollout
Prometheus [19]	Metrics TSDB	Time-series metrics + alert rules	Enrich alerts with path impact + suspects
Grafana [20]	Visualisation	Dashboards + alert routing	Surface topology-aware panels/annotations
OpenNMS [21]	NMS platform	Events + polling + inventory adjacencies	Add topology reasoning + RCA
Kentik [22]	NPM/traffic	Flow-based traffic views	Add correctness/resilience context
Datadog [23] / New Relic [24]	Observability	Logs/metrics/traces + built-in ML	Use as telemetry source; add topology semantics
RPKI [25] + BGPStream [26] (RouteViews/RIS) [27, 28]	Routing security/visibility	Origin validation + global route feeds	Map events to internal reachability/blast radius

1.2.1 How NetAssure Complements the Stack

In practice, operators rarely replace their existing tooling; they add new analysis where the current stack has blind spots.

NetBox / automation. NetAssure can consume inventory and intent from a source-of-truth system (e.g. NetBox) and validate changes proposed by automation (NAPALM/Ansible) before and after rollout.

Monitoring. Prometheus/Grafana/OpenNMS (and commercial observability) answer "what happened" at metric scale. NetAssure adds topology-aware root cause ranking, invariant checking, and resilience impact analysis so alerts can be tied to affected paths and critical components.

Forwarding- and control-plane verification. Dedicated verifiers span both intra- and inter-domain settings, ranging from inline invariant checking to incremental verification under continuous changes. Minesweeper targets correctness of BGP configuration and policy at scale [12]. ARC-style frameworks (placeholder citation) and incremental approaches (e.g. Delta-net) emphasise maintaining invariants efficiently as networks evolve (placeholder citations) [13, 14]. NetAssure is positioned as an internal topology analytics platform; when the scope includes BGP config verification, NetAssure complements (rather than replaces) these specialised verifiers.

Routing security / monitoring. RPKI validation and BGP measurement frameworks (e.g. BGPStream with RouteViews/RIPE RIS feeds) focus on origin/ROA validity, hijack visibility, and macro-level routing events [25–28]. NetAssure focuses on how those control-plane events map onto internal reachability, affected paths, and local remediation actions.

1.2.2 Incrementality and Counterexample Quality

In practice, verification must run continuously as networks evolve: a small change should not force full recomputation. NetAssure emphasises two operator-facing properties: (i) incrementality (recompute only the affected region), and (ii) counterexample quality (return a minimal, actionable witness rather than a long, hard-to-debug trace).

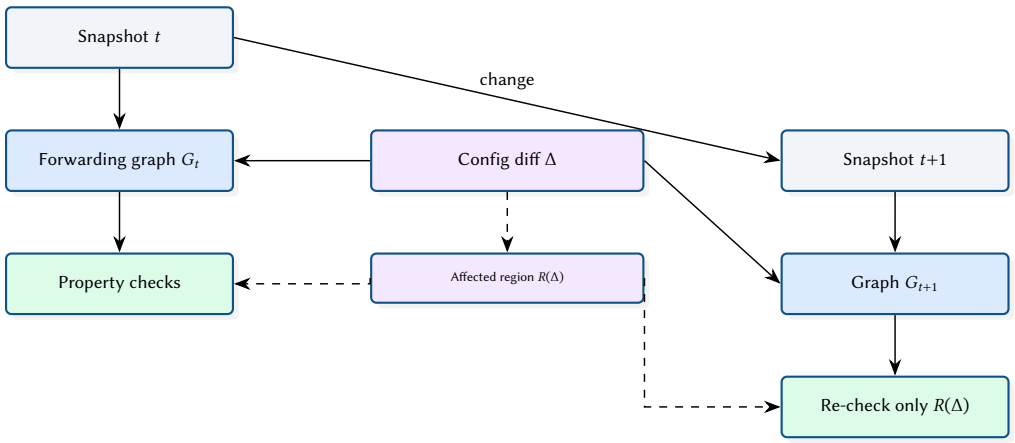


Figure 2. Incremental verification under change: a local configuration diff Δ induces a bounded affected region $R(\Delta)$, enabling reuse of cached forwarding semantics outside the frontier.

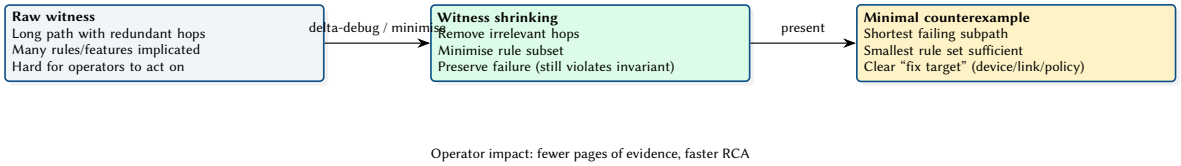


Figure 3. Counterexample quality: shrinking transforms a raw failing witness into a minimal, actionable counterexample that highlights the smallest set of hops and rules sufficient to violate the invariant.

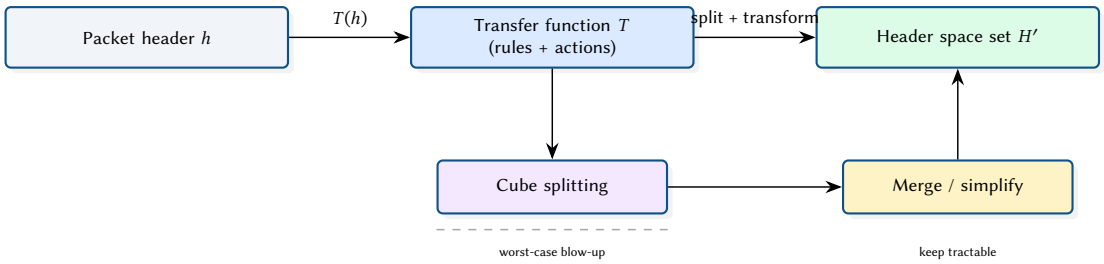
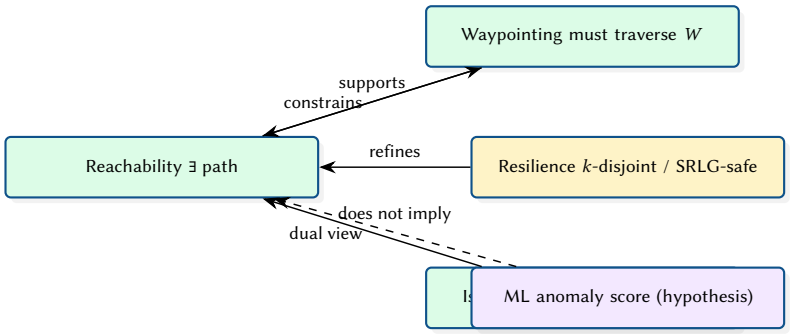


Figure 4. HSA mechanics at a glance: transfer functions map a header space region through match/action rules. Practical engines control split explosion via simplification (merging compatible regions, eliminating unreachable fragments, and caching).



Interpretation: deterministic properties compose into a partial order of strength (constraints refine base reachability). ML signals live outside the guarantee lattice: they prioritise investigation but do not constitute a proof.

Figure 5. Property relationships: verification properties form a lattice of constraints and refinements. Resilience strengthens reachability by requiring diversity; ML scores remain advisory signals that must be reconciled with proofs.

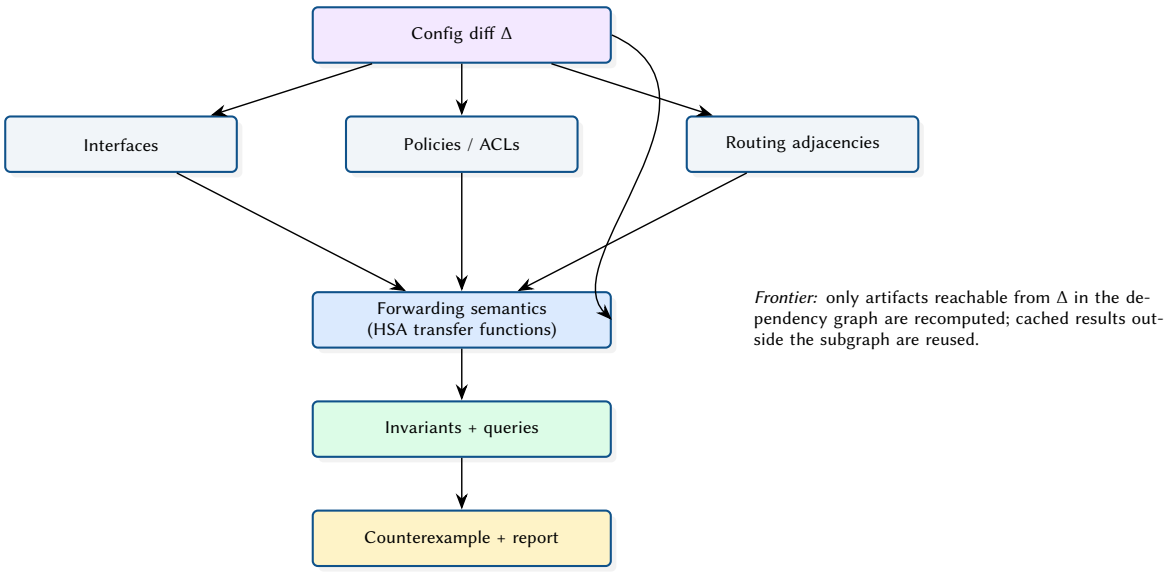


Figure 6. Incremental recompute via dependencies: a change Δ activates a bounded subgraph of dependent artifacts (interfaces, policies, routing adjacencies), which determines the recomputation frontier for forwarding semantics and invariants.

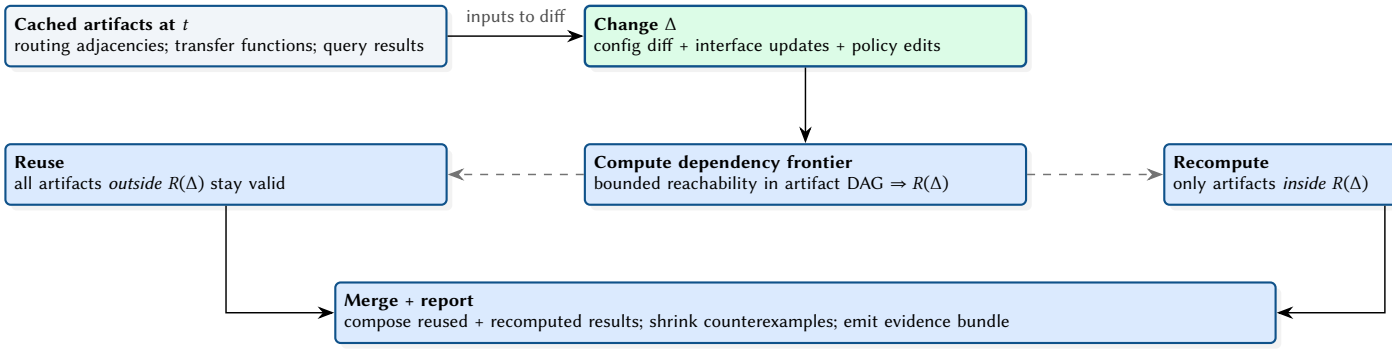
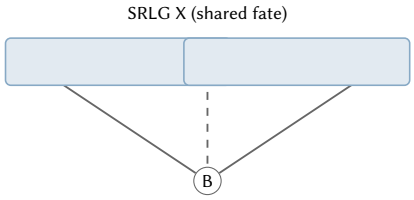
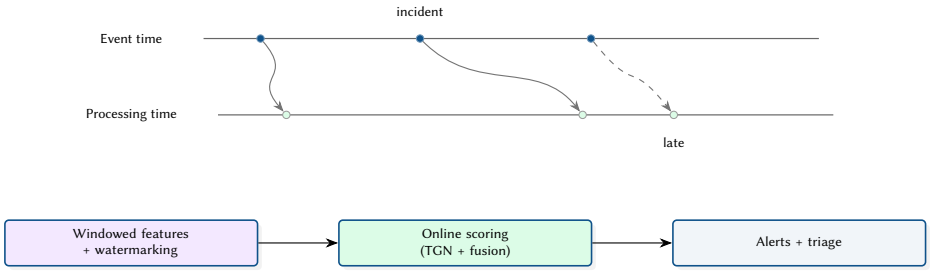


Figure 7. Algorithmic view of incremental verification. The key step is computing a dependency frontier: a bounded region $R(\Delta)$ in the artifact dependency DAG activated by a local change. NetAssure reuses cached semantics outside the frontier and recomputes only inside it, then merges results into a single evidence bundle.



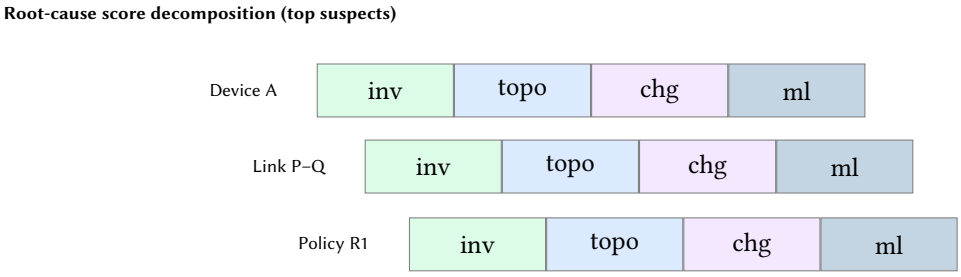
k-disjoint view: two link-disjoint paths $S \to A \to T$ and $S \to B \to T$.
SRLG-aware view: path via A is fragile under SRLG X; diversity is lower than the link model suggests.

Figure 8. SRLG-aware diversity: disjointness under independent-link failures can overestimate resilience when links share risk (common conduit, power domain, or chassis).



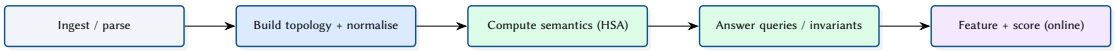
Streaming semantics: NetAssure separates event time from processing time. Watermarks bound lateness; windows define feature aggregation and prevent training/inference leakage.

Figure 9. Streaming semantics: event-time ordering can differ from processing-time arrival due to buffering and delays. Watermarks and windowing make online features well-defined and robust to late data.



Legend: **inv** invariant hits (proof-derived), **topo** topological criticality, **chg** proximity to recent diff, **ml** anomaly likelihood. NetAssure reports both the total score and its components.

Figure 10. Root-cause ranking decomposition: a single suspect score can be expressed as a sum of interpretable components (proof signals, topology, change proximity, and ML likelihood), improving trust and actionability.



Cost model: for steady-state streaming, incremental updates reduce the dominant terms by reusing cached topology and semantics. For batch validation, semantics computation and query evaluation dominate.

Figure 11. A simple cost model for NetAssure: total latency decomposes into ingest/normalisation, topology construction, forwarding semantics, query evaluation, and online ML scoring. Incrementality primarily targets the semantics + query terms.

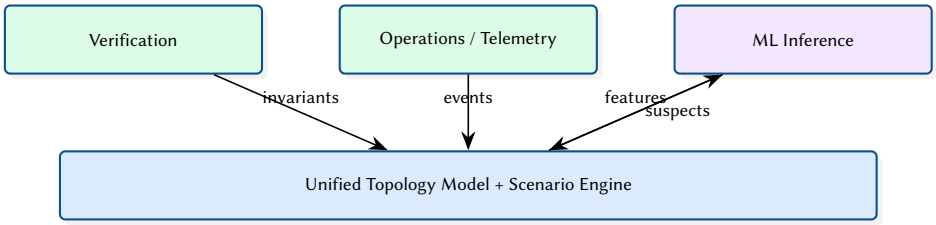


Figure 12. NetAssure's differentiator is the coupling of proofs, telemetry, and predictions through a single scenario engine.

2 System Architecture

2.1 End-to-End Case Study (Single Thread)

To avoid treating each component in isolation, this section provides a single thread that connects the main artefacts the system emits. We use the toy scenario-diff topology in Figure 30 (nodes S,P,Q,T) and interpret the edit as a degraded or removed edge (Q,T). The analysis produces:

1. a verification artefact (counterexample or proof),

2. an impact artefact (scenario diff + blast radius),
3. a resilience artefact (cascade sensitivity), and
4. an ML artefact (ranked suspects + explanation).

Figure 13 summarises the artefacts and their inputs/outputs.

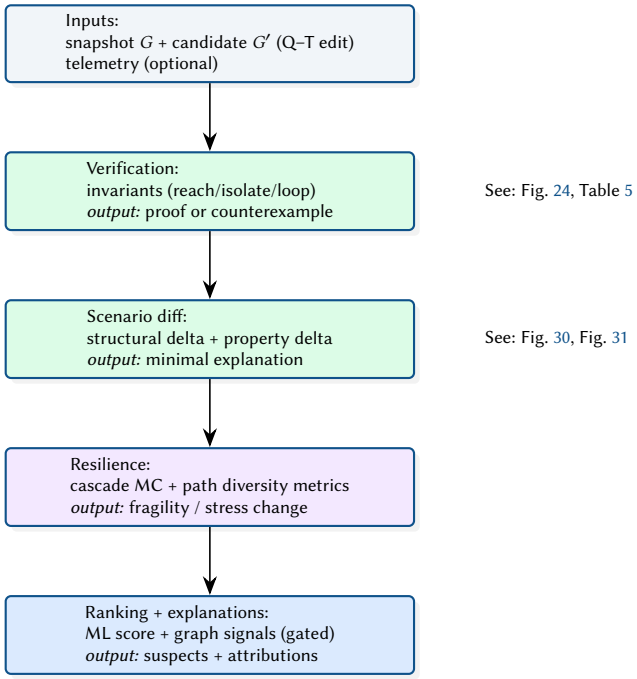


Figure 13. End-to-end case study thread: one topology edit produces verification, impact, resilience, and ML artefacts.

2.2 End-to-End Dataflow

Figure 14 shows how topology data and telemetry flow through the system and where deterministic engines and ML inference sit.

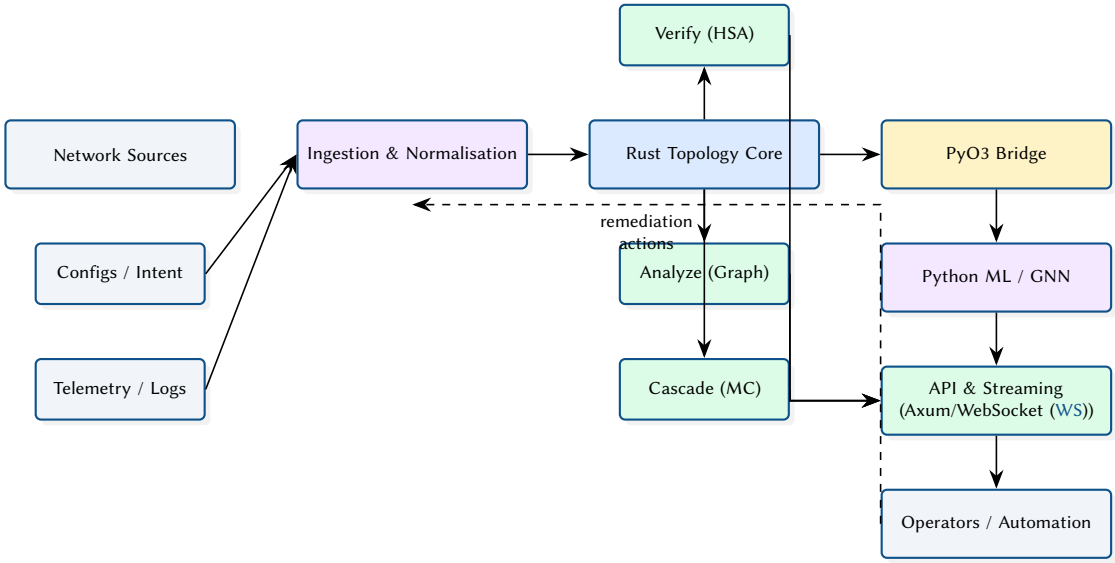


Figure 14. End-to-end dataflow and execution boundaries across Rust and Python.

2.3 Crate Structure

The Rust workspace contains four library crates and a CLI binary:

Table 3. NetAssure Rust crate organisation.

Crate	Role	Key Dependencies
`netassure-core`	Topology model, ingestion	petgraph, serde
`netassure-verify`	HSA, reachability	bit-set, petgraph
`netassure-analyze`	Graph algorithms	rustworkx-core
`netassure-cascade`	Monte Carlo simulation	rayon, rand
`netassure-py`	PyO3 bindings	pyo3
`cli`	Unified CLI	clap, axum, tokio

2.4 Topology Model

The core topology is a `petgraph::StableGraph` with typed node and edge attributes. Stable indices ensure that node/edge removal does not invalidate existing references—critical for cascade simulation where nodes are progressively disabled.

```rust
pub struct Topology {
    graph: StableGraph<NodeAttr, EdgeAttr>,
    node_index: HashMap<String, NodeIndex>,
}

pub struct NodeAttr {
    pub id: String,
    pub node_type: NodeType,
    pub capacity: f64,
    pub load: f64,
}
```

2.5 Data Model: Entities and Relationships

Figure 15 sketches the core entities and relationships in the topology model used across verification, analytics, and cascade simulation.

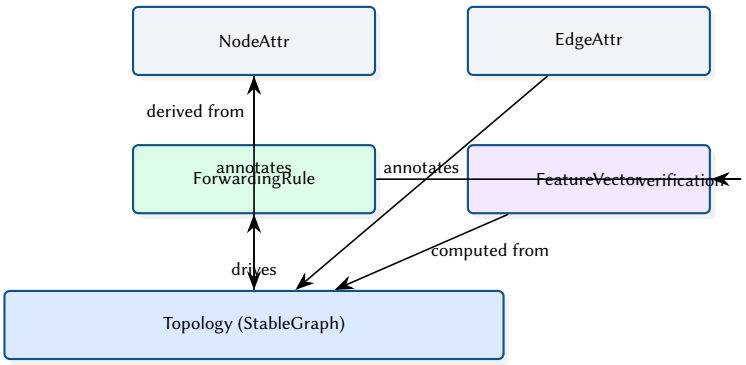


Figure 15. Core topology entities reused by verification, analytics, cascade simulation, and ML feature construction.

2.6 Deployment View

Figure 16 shows a typical deployment split between a Rust service (API + deterministic engines) and a Python inference service, connected through a local boundary.

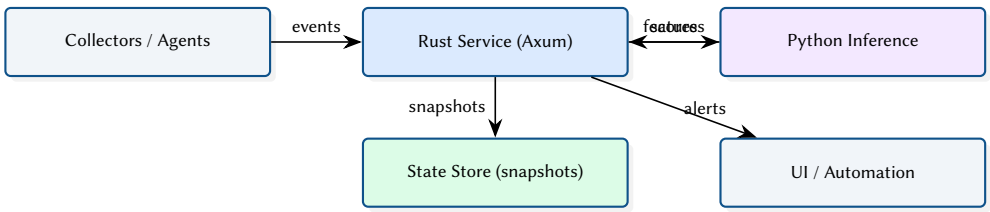


Figure 16. Deployment-oriented view of the Rust API service and Python inference component.

2.7 REST API

The ingestion system exposes an Axum-based HTTP Application Programming Interface (API) for real-time topology synchronisation and prediction queries:

Table 4. REST API endpoints.

Method	Path	Description
GET	`/health`	Liveness probe
GET	`/health/ready`	Readiness (topology synced?)
GET	`/topology`	Full topology JSON
GET	`/predictions`	GNN node predictions
GET	`/anomalies`	Recent anomaly history

2.8 Control Plane vs Data Plane Boundaries

NetAssure keeps a strict separation between what must be correct-by- construction (verification and deterministic analysis) and what is probabilistic (ML inference). Figure 17 sketches the trust boundary used throughout the implementation.

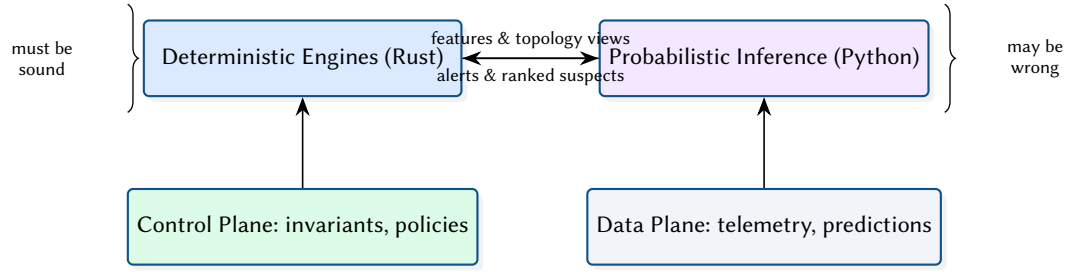


Figure 17. Trust boundary between deterministic analysis and probabilistic inference.

3 Formal Verification

3.1 From Config to Forwarding Semantics

The verifier consumes configuration artefacts (ACLs, routing policy, FIBs) and compiles them into a forwarding semantics used by analysis. Rather than treating the network as a black box, NetAssure makes the intermediate representations explicit: header predicates, actions, and composition. Figure 18 illustrates the concept.

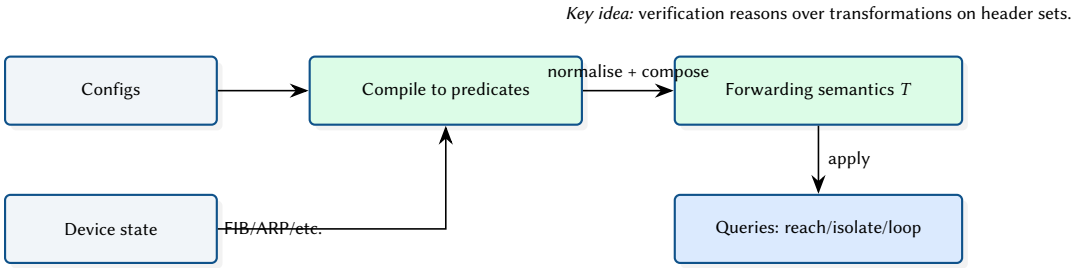


Figure 18. Conceptual compilation pipeline from configuration artefacts to forwarding semantics.

3.2 Header Space Propagation Model

Figure 19 provides a conceptual diagram of header-space propagation across forwarding elements. This abstraction is used to derive reachability, isolation violations, and loop candidates.

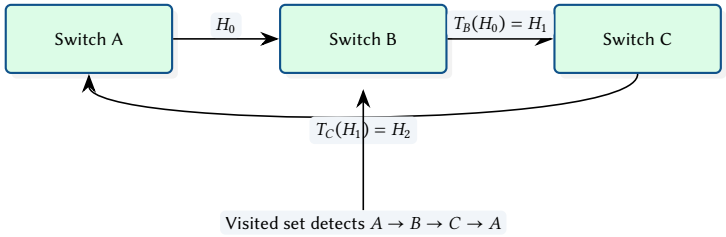


Figure 19. Conceptual header-space propagation and loop detection.

3.3 Fixpoint View: Termination and Over-Approximation

Operationally, reachability queries are solved by repeated propagation until a fixpoint is reached: no new (node,port,header-set) states are discovered. To guarantee termination, implementations use canonical representations (e.g. BDDs) and may apply widening/merging that produces safe over-approximations. Figure 20 illustrates the concept.

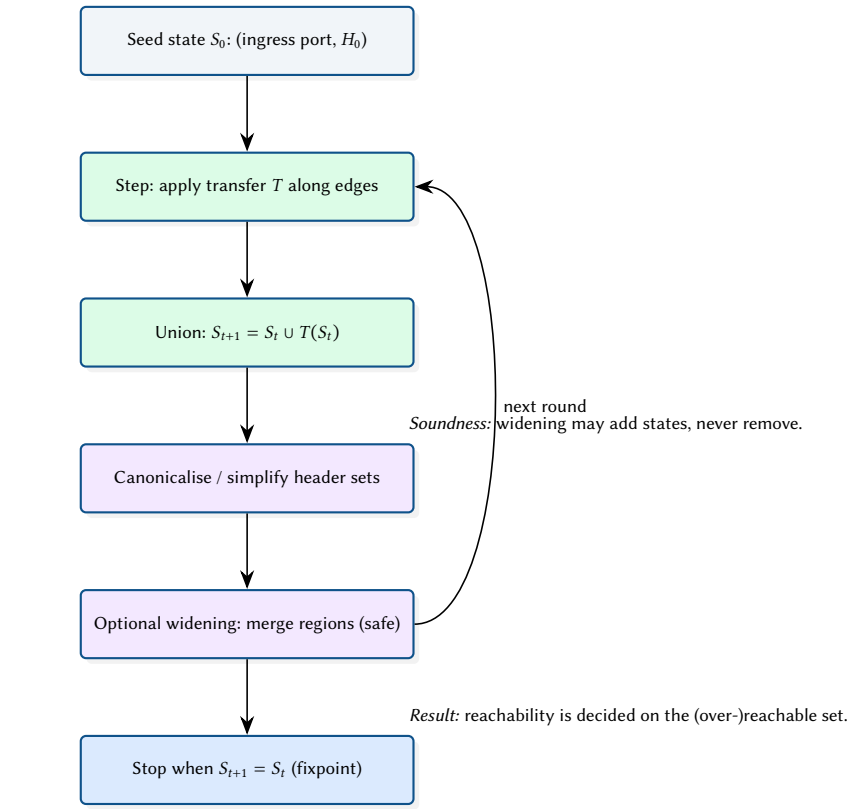


Figure 20. Fixpoint interpretation of header-space propagation with canonicalisation and safe widening.

3.4 Header Space as Geometry (Concept Diagram)

Header space can be visualised as a high-dimensional cube. Predicates carve that space into regions, and forwarding actions map regions from one interface to another. Figure 21 illustrates this idea with a 2D slice (for intuition).

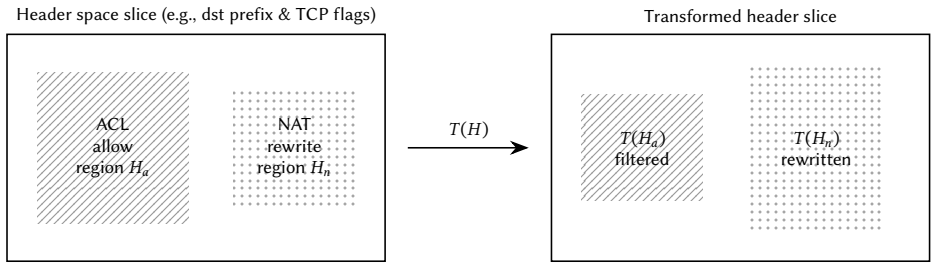
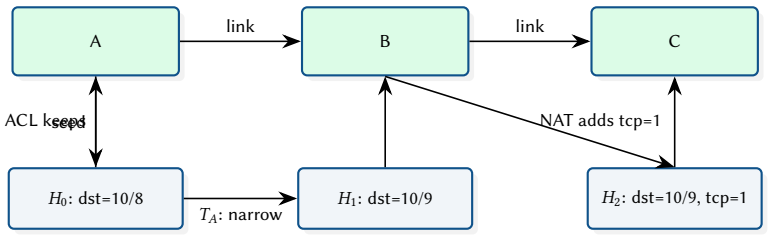


Figure 21. HSA intuition: header predicates carve space into regions; forwarding maps regions across interfaces.

3.5 Worked Reachability Example (Region Propagation)

The following toy example shows how a reachability query propagates a small set of header regions across hops. Each device applies a filter (e.g. ACL) and a transform (e.g. rewrite), producing new regions. The verifier maintains a set of discovered states and iterates until no new regions are produced.



Interpretation: regions become more specific as constraints accumulate; implementation canonicalises/merges to keep the set finite.

Figure 22. Toy propagation of three header regions across three hops for a reachability query.

3.6 Verification Query Types

Operators typically run a small set of repeatable questions. The verification engine exposes these as query templates (Figure 23).

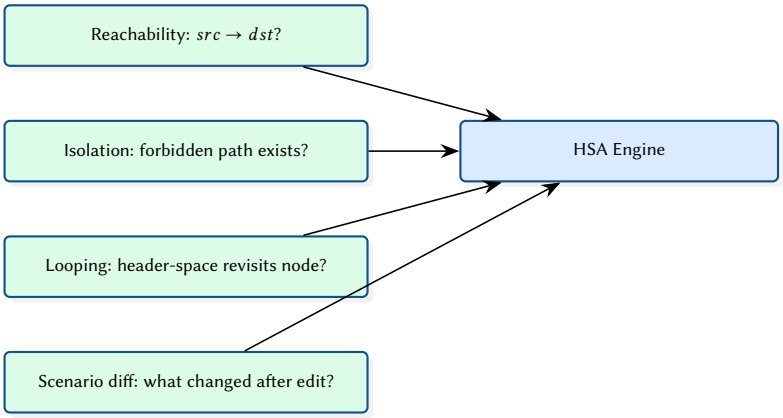


Figure 23. Common verification query templates exposed to operators and automation.

3.7 Invariant Catalog (Operator View)

NetAssure exposes verification as a small catalog of invariants. Each invariant produces either a proof of satisfaction (often implicit) or a concrete counterexample (path + header region) when violated. Table 5 summarises representative invariants and the artefacts returned to operators and automation.

Table 5. Representative invariant catalog and operator-facing artefacts.

Invariant	Meaning	Common break	Artefact on violation
Reachability	$src \rightarrow dst$ exists	missing route / ACL drop	counterexample path + header predicate
Isolation	forbidden $src \rightarrow dst$ absent	overly-permissive ACL/NAT	violating path + minimal header region
No loops	no cyclic forwarding for any H	recursive route / policy bug	loop trace + header region inducing it
Waypointing	traffic must traverse middlebox	asymmetric policy	shortest violating bypass path
ECMP consistency	path set respects intended hashing	unequal next-hop sets	diverging next-hop witness
Blackhole-free	no sink states for allowed H	null route / missing ARP	sink node + header region
MTU-safety	headers fit along path	MTU mismatch / tunneling	first-hop MTU witness
Latency bound	any feasible path meets budget	detours after failure	critical edge/cutset causing detour

3.8 Counterexample Output (Path + Header Region)

When an invariant fails, the most valuable output is a small, concrete counterexample. Figure 24 illustrates the intended shape: a violating path annotated with the header predicate that makes the failure possible.

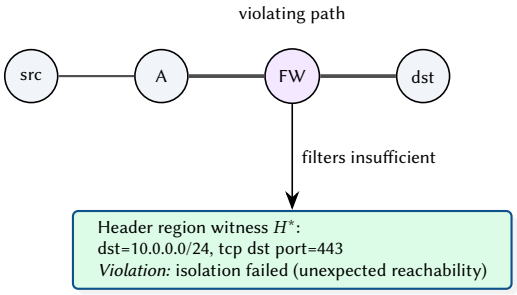


Figure 24. Counterexample concept: a small violating path plus a minimal header predicate witness.

The verification engine implements HSA [1] over the topology graph. Each forwarding rule is modelled as a header-space transfer function; the engine computes reachability by composing transforms along all paths between source and destination.

> **Design Decision 1: Bit-Set Header Representation**
>
> Header spaces are represented as bit-sets rather than ternary vectors. This trades some expressiveness for $O(1)$ intersection and union operations, which dominate the verification runtime. For the network sizes targeted (hundreds of nodes, thousands of rules), this approach is sufficient and avoids the complexity of BDD-based representations.

Loop detection operates by tracking visited nodes during header-space propagation. If a packet's header matches a forwarding rule that would send it to an already-visited node, a loop is reported with the full cycle path.

4 Failure Cascade Simulation

4.1 Cascade Dynamics

Figure 26 illustrates the round-based cascade loop: initial failures remove capacity, load redistributes, and newly overloaded nodes fail until convergence.

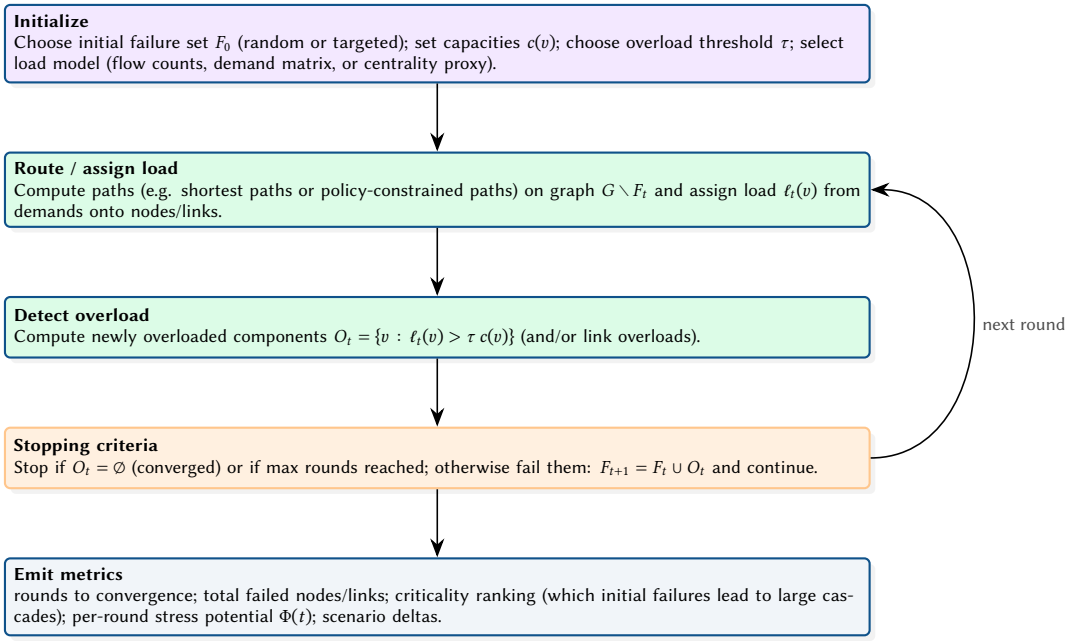


Figure 25. Cascade Monte Carlo algorithm as a deterministic inner loop wrapped by randomised initial conditions. The key design choice is the load model: it must be cheap enough to run for thousands of trials, but faithful enough to reproduce rerouting-induced overload cascades.

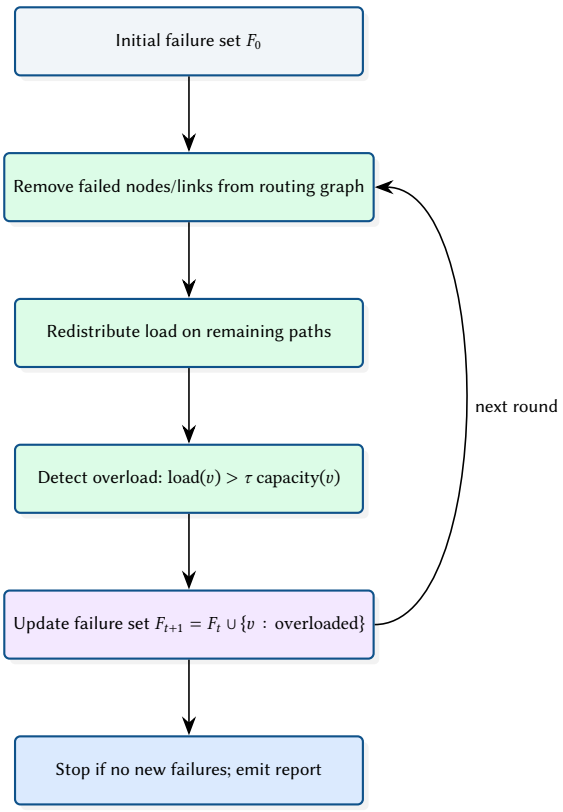


Figure 26. Round-based cascade progression used in the Monte Carlo simulator.

4.2 Toy Example: Rerouting Amplifies Load

To make the cascade mechanics tangible, Figure 27 shows a small topology where an initial link failure forces rerouting onto a longer path. The redistributed load may exceed capacity on a previously non-critical node, creating a secondary failure.

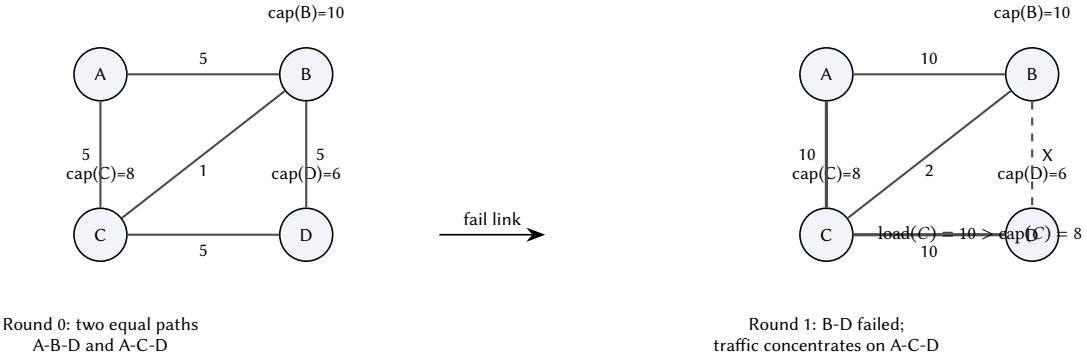


Figure 27. Toy cascade: rerouting after a single failure can overload a different component and trigger secondary failures.

4.3 Load Redistribution as a Potential Function

The cascade loop can be interpreted as a sequence of constrained re-routing steps. A helpful mental model is that each round reduces the feasible routing set, increasing the "stress" on remaining components. Figure 28 visualises this as a potential function over rounds.

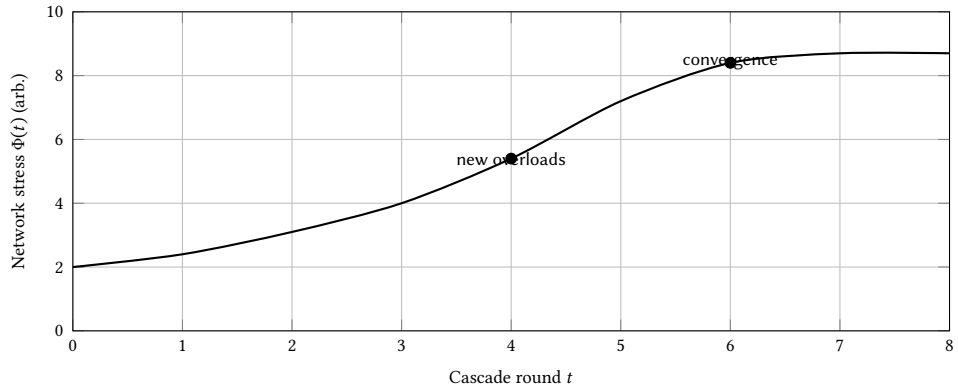


Figure 28. Conceptual view: as failures remove options, stress increases until the cascade converges.

4.4 Scenario Comparison Output

When comparing two topologies (e.g. baseline vs. redundancy upgrade), NetAssure emits paired distributions and deltas. Figure 29 illustrates the intended output.

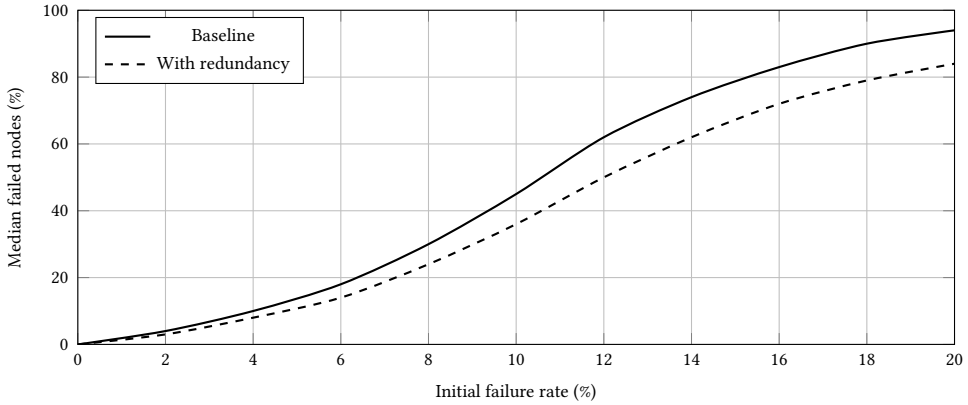
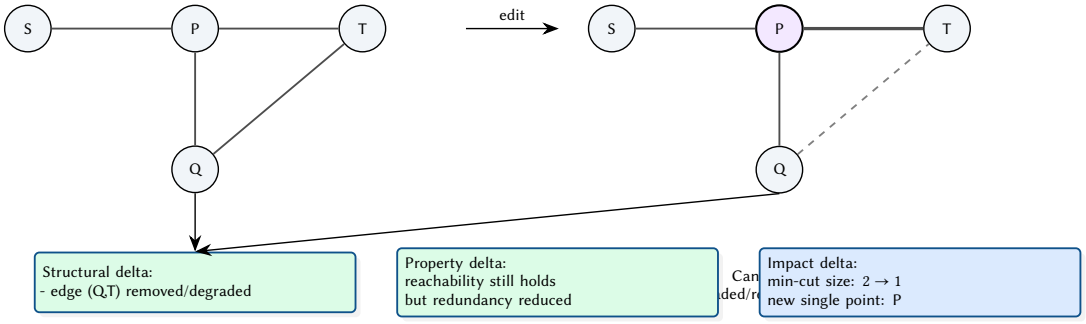


Figure 29. Illustrative scenario comparison: redundancy reduces cascade scope across failure rates.

4.5 Scenario Diff Semantics: What Changed and Why It Matters

A scenario diff compares a baseline topology G and a candidate topology G'. NetAssure computes (i) structural deltas (added/removed nodes/links), (ii) property deltas (reachability/invariant changes), and (iii) impact deltas (affected paths and changes in resilience). The goal is a *minimal explanation* for the observed behavioural change. Figure 30 illustrates this concept.



Minimal explanation: one edge edit collapses path diversity, increasing blast radius.

Figure 30. Scenario diff semantics: structural changes drive property changes, which drive operational impact (paths, cuts, resilience).

4.6 Blast Radius View: Which Flows Lose Redundancy

Operators care about which services and demand pairs are impacted by a change. A structural edit may preserve reachability but reduce the number of disjoint paths for a subset of flows, increasing the blast radius

under subsequent failures. Figure 31 shows a toy example where one demand pair loses path diversity after an edit.

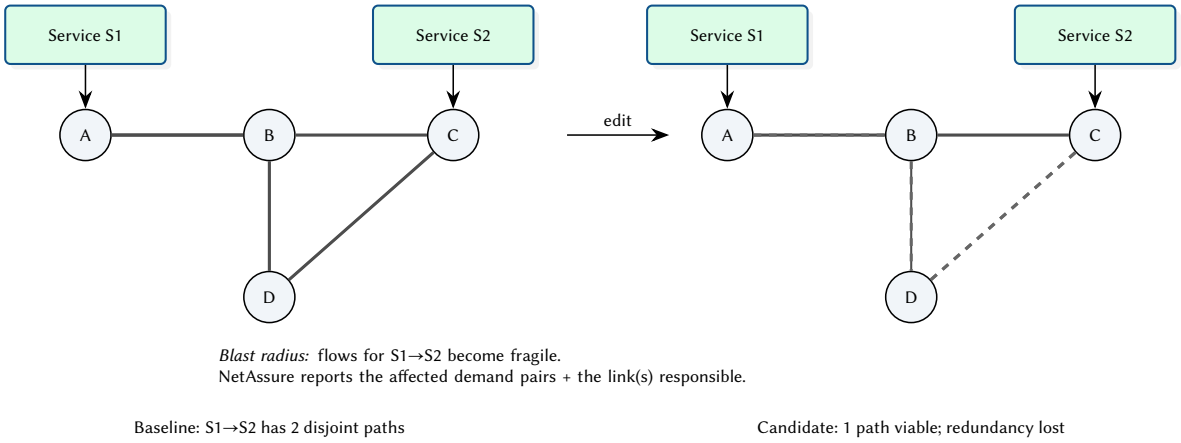


Figure 31. Blast radius concept: a change can reduce path diversity for specific services, increasing risk even if reachability remains.

4.7 Blast Radius Report (Illustrative Output)

Beyond the conceptual view, operators benefit from a compact report that names the most impacted demand pairs/services and the structural causes. Table 6 shows an illustrative format used by NetAssure: for each demand pair, report reachability and an estimate of path diversity before/after the scenario change, along with the critical edge(s) responsible for the drop. To keep the narrative consistent, the rows use the same toy node names introduced in Figure 30 (S,P,Q,T) and Figure 31 (A,B,C,D).

Table 6. Illustrative blast radius report for a scenario diff (synthetic example).

Demand pair	Service	Reachable	k disjoint	Critical edge(s)	Notes
A→C	svc-A→svc-C	yes	2 → 1	(D,C)	Matches Figure 31: link D–C degraded/removed
A→D	svc-A→svc-D	yes	2 → 2	–	No diversity change in this toy edit
S→T	svc-S→svc-T	yes	2 → 1	(Q,T)	Matches Figure 30: redundancy collapses onto P
S→Q	svc-S→svc-Q	yes	1 → 1	–	Still depends on P but unchanged under the edit

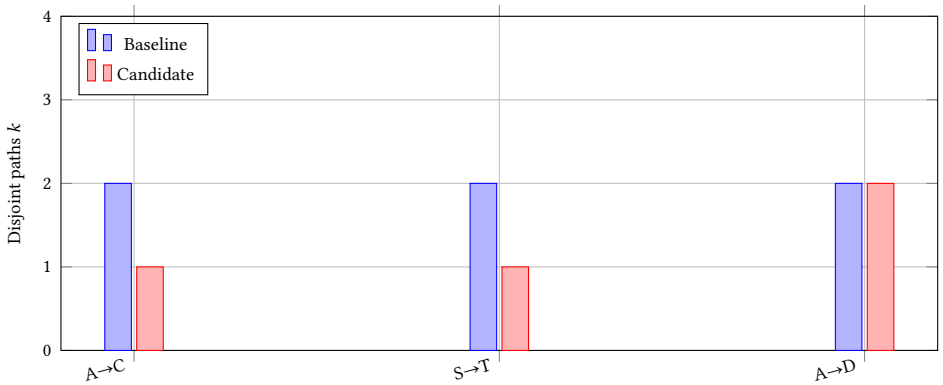


Figure 32. Illustrative summary plot: path diversity k for selected demand pairs before/after a scenario change.

4.8 Example Resilience Curve

The Monte Carlo simulator produces distributions of cascade scope. Figure 33 shows an illustrative curve of median failed fraction vs. initial failure rate.

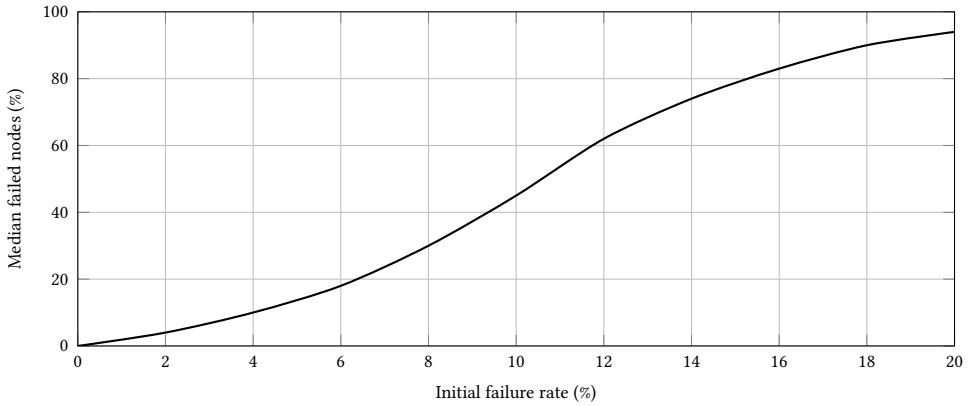


Figure 33. Illustrative resilience curve from Monte Carlo cascade simulation.

The cascade simulator models progressive network failure through load redistribution. When a node or link fails, its traffic is redistributed across remaining paths. If any node exceeds its capacity threshold, it fails in turn, potentially triggering a cascade.

```rust
pub fn simulate_cascade(
    topo: &mut Topology,
    initial_failures: &[NodeIndex],
    redistribution: RedistributionPolicy,
    capacity_threshold: f64,
) -> CascadeReport {
    let mut round = 0;
    let mut failed: HashSet<NodeIndex> = initial_failures.iter().cloned().collect();
    loop {
        let overloaded = find_overloaded(&topo, &failed, capacity_threshold);
        if overloaded.is_empty() { break; }
        failed.extend(overloaded);
        redistribute_load(topo, &failed, &redistribution);
        round += 1;
    }
    CascadeReport { rounds: round, total_failed: failed.len(), .. }
}
```

Monte Carlo mode runs thousands of iterations with random initial failure sets (parameterised by failure percentage), using Rayon for parallel execution. Each iteration produces a cascade depth and scope, enabling statistical analysis of network resilience.

5 Machine Learning Pipeline

6 Complexity, Scaling, and Incrementality

This section documents what drives runtime and memory use. The dominant factors are (i) topology size, (ii) policy/rule volume, and (iii) the number of distinct header regions produced by propagation.

Insight:
Scenario comparison mode runs two cascade experiments with different topologies (e.g. before and after adding redundancy) and produces a diff report highlighting improvements in cascade depth, scope, and critical node rankings.

6.1 What Drives Cost

Figure 34 highlights the main cost drivers and which steps are incremental vs full recompute.

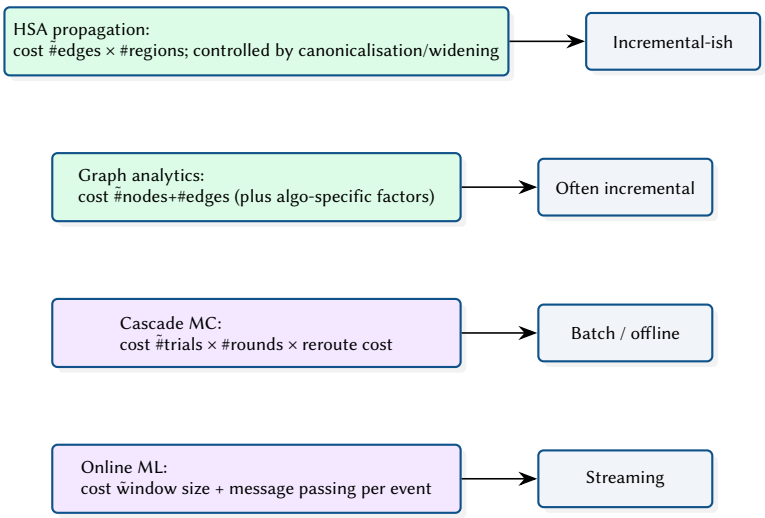


Figure 34. Scaling drivers across verification, analytics, simulation, and ML.

6.2 Incremental Update Path (Concept)

When processing a diff, NetAssure aims to recompute only what changed: localised graph updates, affected forwarding semantics, and impacted demand pairs. Figure 35 illustrates the incremental path.

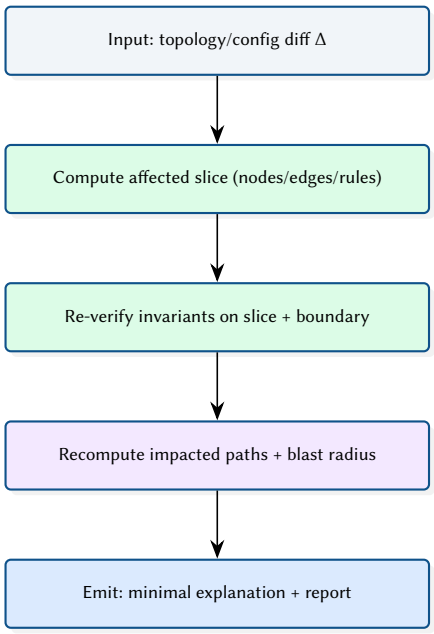


Figure 35. Conceptual incremental update path: restrict recomputation to an affected slice when analysing diffs.

7 Limitations and Failure Modes

NetAssure's outputs are only as good as the inputs and assumptions. This section describes common failure modes and how the system surfaces uncertainty.

7.1 Incomplete Inventory and Stale Snapshots

Figure 36 illustrates a typical failure mode: inventory says a link exists, but the observed state has drifted. The system should present this as uncertainty rather than a false proof.

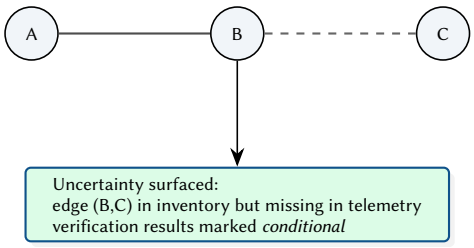


Figure 36. Failure mode: stale/incomplete snapshots. Missing edges should downgrade proofs to conditional results.

7.2 Ambiguous Config Semantics

Some devices and vendors have semantics that are hard to model (e.g. route-map corner cases, implicit defaults). The recommended behaviour is to report the modeling assumptions explicitly and provide conservative over-approximations.

7.3 ML Uncertainty and Concept Drift

ML models degrade under drift. NetAssure should expose drift indicators and avoid presenting probabilistic outputs as proofs.

8 Visual Language and Diagram Conventions

To keep diagrams consistent, NetAssure uses the following conventions: solid edges are present/healthy, dashed edges are degraded/removed, highlighted nodes indicate suspicious/critical components, and pattern fills represent sets/regions rather than concrete packets.

8.1 Online Inference and Alerting

Figure 37 sketches the online loop used by the real-time detector: ingest events, update features, run inference, and push alerts to operators.

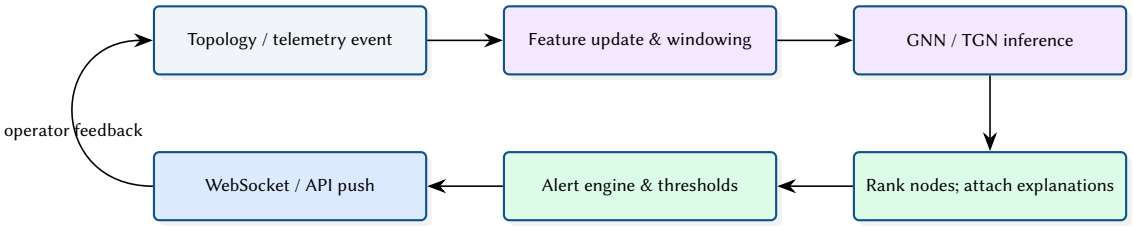


Figure 37. Online inference and alerting pipeline.

8.2 Temporal Message Passing (Concept Diagram)

Temporal models consume events on edges and update node memories over time. Figure 38 illustrates a single event update: messages are computed from source/target state and event features, then aggregated into memory.

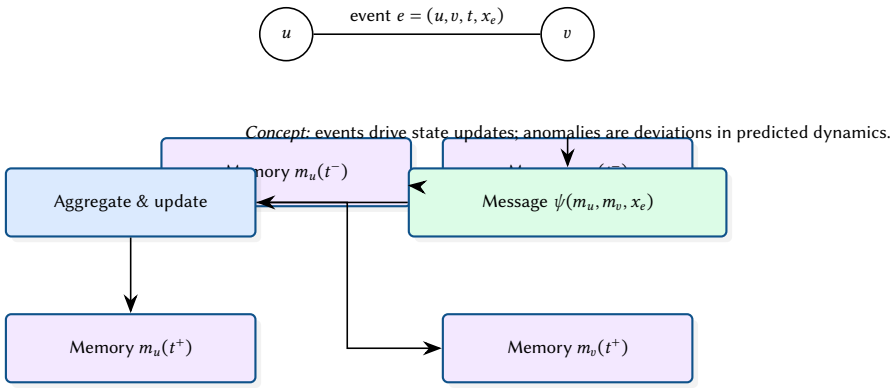


Figure 38. Temporal message passing concept (TGN-like): event features update node memory through messages and aggregation.

8.3 Hybrid Decision: Hard Constraints + Ranked Hypotheses

NetAssure combines deterministic assertions (must-hold invariants) with probabilistic signals (ranked hypotheses). Figure 39 illustrates the decision logic: hard failures trigger immediate violations, while ML scores prioritise investigation among the remaining feasible explanations.

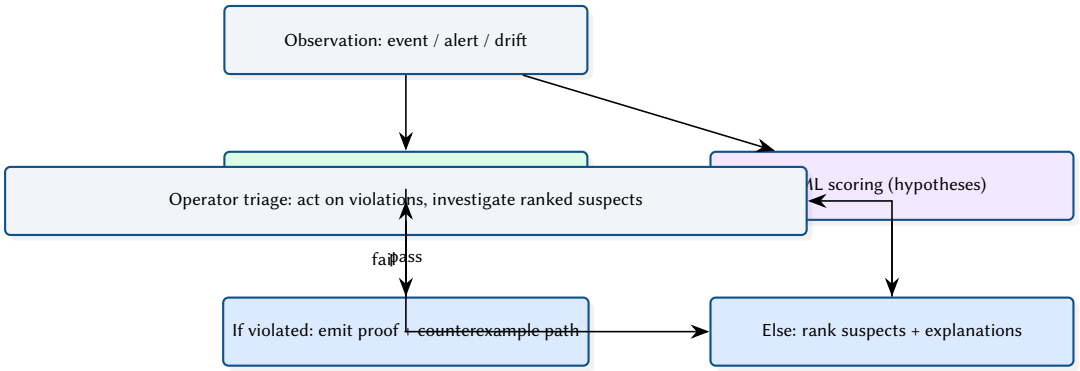


Figure 39. Hybrid decision model: deterministic checks gate correctness; ML prioritises investigation within the feasible space.

8.4 GNN Anomaly Detection

The GNN model [7] operates on a 14-dimensional feature schema per node (degree centrality, betweenness, load ratio, interface counts, etc.) and produces per-node anomaly scores.

8.5 Model Families and Trade-offs

Different model families offer different operational trade-offs. Figure 40 shows an illustrative comparison of latency and operator interpretability (synthetic values).

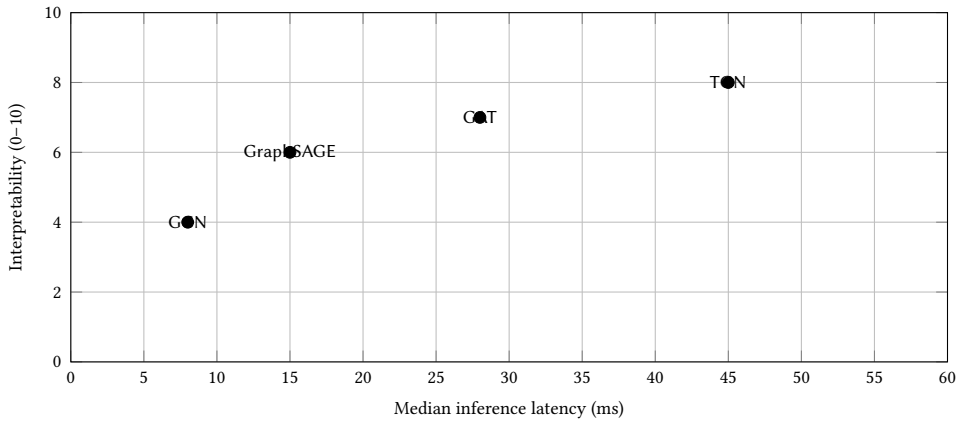
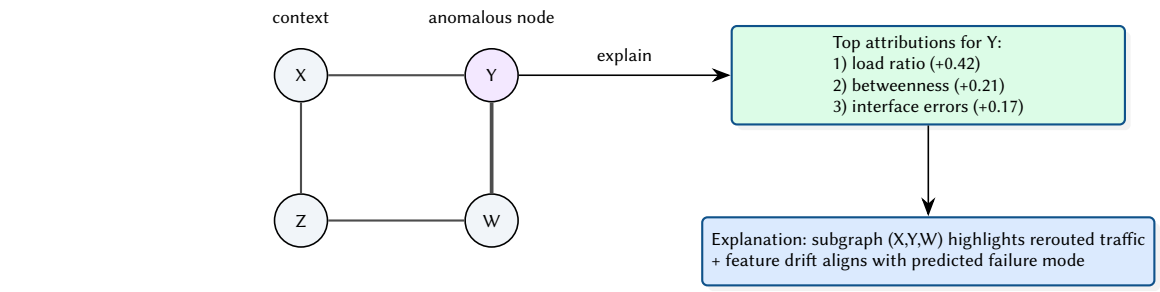


Figure 40. Illustrative trade-off plot: latency vs. interpretability across model families.

8.6 Explanation Output: Subgraph + Feature Attribution (Concept)

To support operator trust, NetAssure presents an anomaly score together with an explanation: a small subgraph and the most influential features. Figure 41 illustrates the intended output format.



Interpretation: show *where* (subgraph) and *why* (features), not just a score.

Figure 41. Conceptual explanation output: anomalous node with a small explanatory subgraph and feature attributions.

8.7 Temporal Graph Networks

The Temporal Graph Network (TGN) model processes time-series topology events (link flaps, load changes) to predict future anomalies. It maintains a per-node memory vector updated at each event, enabling the model to capture temporal patterns such as recurring failures or gradual degradation.

8.8 Multi-Modal Fusion

The fusion scorer combines three signal types:

- **Topology embeddings** (14-dim feature schema)
- **Metrics sequence** (5-dim: latency, loss, jitter, throughput, error rate)
- **Log embeddings** (768-dim from a pre-trained encoder)

> **Design Decision 2: Fusion Architecture**
>
> The fusion model concatenates the three embedding vectors after per-modality projection heads, then passes through a two-layer MLP with dropout. This late-fusion approach allows each modality to be independently pre-trained and evaluated, simplifying debugging when one signal source is noisy or unavailable.

8.9 Fusion Model (Concept + Math)

NetAssure uses deterministic checks as gates and ML as ranking. One simple formulation is a constrained ranking score:

$$\text{score}(v) = \mathbb{I}[\neg\text{violates}(v)] \left(\alpha\, s_{\text{ml}}(v) + (1-\alpha)\, s_{\text{graph}}(v)\right) \tag{1}$$

where s_{ml} is the learned anomaly score, s_{graph} encodes deterministic signals (e.g. centrality, load ratio, proximity to affected paths), and $\mathbb{I}$ gates out nodes that already have a hard violation.

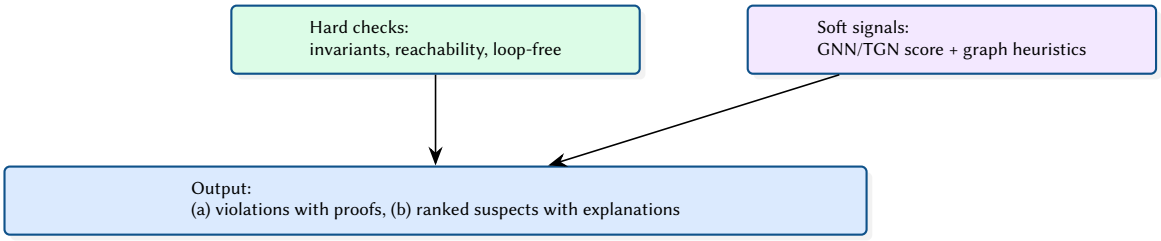


Figure 42. Fusion concept: correctness gates (hard) plus prioritisation signals (soft).

8.10 Graph Metrics to Operational Meaning (Concept)

Graph analytics provide deterministic, interpretable signals that can be used directly (e.g. to find choke points) and as features for ML. Figure 43 illustrates how common metrics map to operator questions.

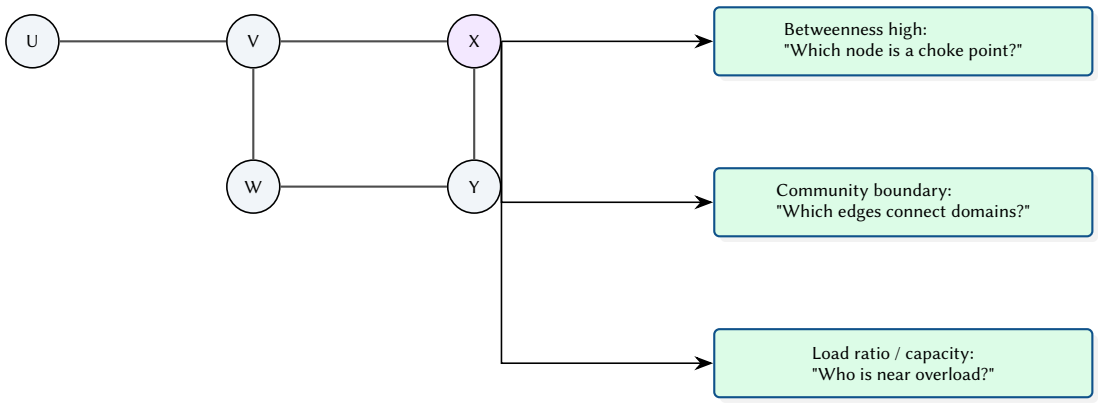


Figure 43. Conceptual mapping from graph metrics to operational questions and interpretable signals.

8.11 End-to-End Evaluation Protocol

Figure 44 summarises a practical evaluation protocol for combined deterministic and ML components: validate invariants, then evaluate detection quality and operational cost.

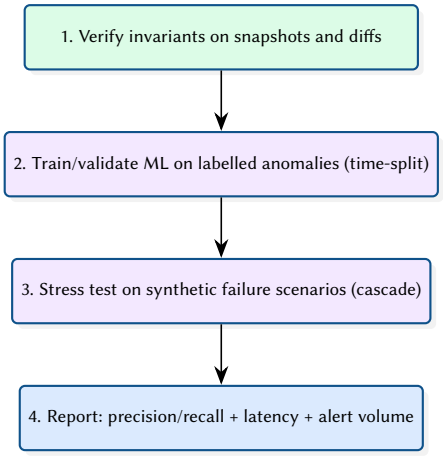


Figure 44. Evaluation protocol across verification, simulation, and ML inference.

8.12 Evaluation Plan: Offline vs Online Loops

Even without a full production dataset, the evaluation section should separate what can be validated offline (correctness and ML quality) from what must be validated online (latency, alert volume, operator utility). Figure 45 sketches these two loops.

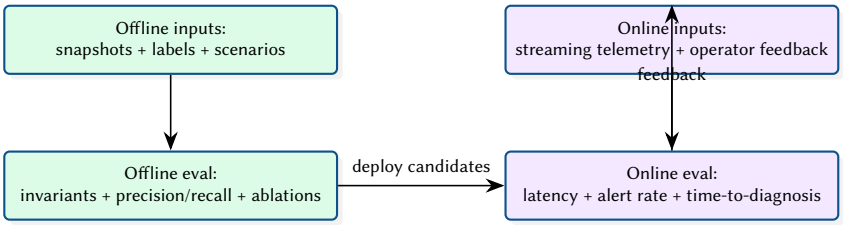


Figure 45. Evaluation concept: offline verification/ML metrics vs online operational metrics.

9 Design Summary

Table 7. Key design decisions and their rationale.

Decision	Rationale
Hybrid Rust/Python	Deterministic algorithms in Rust for speed; ML in Python for ecosystem maturity
PyO3 bindings	Zero-copy where possible; avoids JSON serialisation on the hot path
StableGraph	Node removal during cascade simulation must not invalidate existing indices
Bit-set HSA	$O(1)$ set operations; sufficient for target network sizes
Late fusion scoring	Independent modality training; graceful degradation when a signal is missing

References

[1] P. Kazemian, G. Varghese, and N. McKeown, "Header space analysis: Static checking for networks," *NSDI*, 2012.

[2] P. Kazemian, R. Govindan, and G. Varghese, "Real time network policy checking using header space analysis," in *NSDI*, 2013.

[3] A. Khurshid, W. Zhou, M. Caesar, and P. B. Godfrey, "VeriFlow: Verifying network-wide invariants in real time," in *NSDI*, 2013.

[4] A. Panda, O. Lahav, K. Lavi, S. Itzhaky, M. Schroeder, and S. Shenker, "Verifying reachability in networks with Batfish," in *NSDI*, 2015.

[5] C. Anderson et al., "NetKAT: Semantic foundations for networks," in *POPL*, 2014.

[6] A. E. Motter and Y.-C. Lai, "Cascade-based attacks on complex networks," *Physical Review E*, 2002.

[7] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *ICLR*, 2017.

[8] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," *NeurIPS*, 2017.

[9] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *ICLR*, 2018.

[10] E. Rossi, B. Chamberlain, F. Frasca, D. Eynard, F. Monti, and M. M. Bronstein, "Temporal graph networks for deep learning on dynamic graphs," *arXiv*, 2020. [Online]. Available: https://arxiv.org/abs/2006.10637

[11] R. Ying, D. Bourgeois, J. You, M. Zitnik, and J. Leskovec, "GNNExplainer: Generating explanations for graph neural networks," in *NeurIPS*, 2019.

[12] R. McNutt, A. Panda, M. Budiu, N. McKeown, and S. Shenker, "Minesweeper: An efficient verification tool for BGP configurations," in *NSDI*, 2020.

[13] *ARC: A unified platform for network verification and synthesis*, SIGCOMM, Placeholder citation. Replace with canonical BibTeX (authors/doi/pages) when available., 2016.

[14] *Delta-net: Real-time network verification using network change histories*, SIGCOMM, Placeholder citation. Replace with canonical BibTeX (authors/doi/pages) when available., 2019.

[15] NetBox Project, *NetBox: Infrastructure resource modeling*, 2026. [Online]. Available: https://github.com/netbox-community/netbox

[16] NAPALM Community, *NAPALM: Network automation and programmability abstraction layer with multivendor support*, 2026. [Online]. Available: https://github.com/napalm-automation/napalm

[17] K. Byers and contributors, *Netmiko: Multi-vendor SSH library*, 2026. [Online]. Available: https://github.com/ktbyers/netmiko

[18] Red Hat, *Ansible: Automation platform*, 2026. [Online]. Available: https://www.ansible.com

[19] Prometheus Authors, *Prometheus: Monitoring system and time series database*, 2026. [Online]. Available: https://prometheus.io

[20] Grafana Labs, *Grafana: Observability and data visualization*, 2026. [Online]. Available: https://grafana.com/grafana/

[21] OpenNMS Group, *OpenNMS: Network management platform*, 2026. [Online]. Available: https://www.opennms.com

[22] Kentik, *Kentik: Network observability and traffic analytics*, 2026. [Online]. Available: https://www.kentik.com

[23] Datadog, *Datadog: Monitoring and security platform*, 2026. [Online]. Available: https://www.datadoghq.com

[24] New Relic, *New relic: Observability platform*, 2026. [Online]. Available: https://newrelic.com

[25] RIPE NCC and IETF Community, *RPKI: Resource public key infrastructure*, 2026. [Online]. Available: https://rpki.readthedocs.io

[26] CAIDA, *BGPStream: A software framework for live and historical BGP data analysis*, 2026. [Online]. Available: https://bgpstream.caida.org
[27] University of Oregon, *Routeviews project*, 2026. [Online]. Available: https://www.routeviews.org
[28] RIPE NCC, *RIPE RIS: Routing information service*, 2026. [Online]. Available: https://www.ripe.net/analyse/internet-measurements/routing-information-service-ris

List of Figures

List of Tables

List of Design Decisions & Rules

Revision History

Version	Date	Changes
0.1.0	March 2026	Initial architecture report